



KARGO YÖNETİM SİSTEMİ

Kargo Yönetim Sistemi Proje Raporu

Hazırlayanlar:

Mustafa Deniz Çam - 170424054

Elif Biçan - 100921017

Talha Sağırlı-171424015

Ders: Veri Yapıları ve Algoritmalar

Proje Konusu: Kargo Yönetim Sistemi

Geliştirme Platformu: Java SE & Java Swing GUI

Tarih: 19 Mayıs 2026

İçindekiler

1. Giriş ve Proje Amacı	3
2. Sistem Mimarisi ve Sınıf Tasarımları	3
2.1. Veri Modelleri (Entity Classes)	3
2.2. Yönetici ve İş Mantığı Sınıfları	4
3. Kullanılan Veri Yapıları ve Teorik Altyapı	4
3.1. Hash Table (Karma Tablosu) Altyapısı	4
3.2. FIFO Queue ve Priority Queue Entegrasyonu	5
3.3. Çizge (Graph) Veri Yapısı ve Komşuluk Listesi	5
4. Uygulanan Algoritmalar ve Optimizasyonlar	6
4.1. Çok Kriterli Dijkstra Algoritması	6
4.2. First Fit Decreasing (Bin Packing) Optimizasyonu	6
5. Grafik Kullanıcı Arayüzü (GUI) ve Modül İncelemeleri	7
6. Geliştirme Sürecinde Karşılaşılan ve Giderilen Zorluklar	8
7. Gelecekte Geliştirilebilecek Yerler ve Öneriler	9
8. Sonuç	9
9. Kaynakça	10

1. Giriş ve Proje Amacı

Modern lojistik ve tedarik zinciri yönetimi, büyük hacimli verilerin eş zamanlı olarak işlenmesini, operasyonel maliyetlerin minimize edilmesini ve zaman hassasiyeti yüksek teslimatların optimize edilmesini zorunlu kılmaktadır. Elle yapılan planlamalar ve sezgisel yaklaşımlar, düğüm (şehir) ve paket sayısı arttıkça yetersiz kalmakta, bu durum operasyonel zararlara yol açmaktadır.

Bu projenin temel amacı; bir kargo şirketinin paket kabulünden nihai dağıtım ve rota planlamasına kadar olan tüm süreçlerini optimize eden, bilgisayar bilimlerinin temel veri yapıları ve algoritmalarını harmanlayan nesne yönelimli bir **Kargo Yönetim Sistemi** simülatörü geliştirmektir. Proje kapsamında rastgele veya doğrusal veri araması yerine anlık erişim mimarileri, standart FIFO mekanizmaları yerine VIP önceliklendirme matrisleri, en kısa yol problemleri ve kısıtlı alanlara maksimum nesne yerleştirme algoritmaları entegre bir biçimde uygulanmıştır.

2. Sistem Mimarisi ve Sınıf Tasarımları

Sistem, sürdürülebilir, modüler ve genişletilebilir bir mimari sağlamak amacıyla Nesne Yönelimli Programlama (OOP) prensiplerine tam uyumlu şekilde tasarlanmıştır. Sistemdeki sınıflar "Veri Modelleri" ve "İş Mantığı / Algoritma Yöneticileri" olmak üzere iki temel katmanda toplanmaktadır.

2.1. Veri Modelleri (Entity Classes)

Sistemde fiziksel nesneleri ve lojistik kavramları temsil eden dört temel veri modeli bulunmaktadır:

- **Package.java:** Gönderinin kimlik (ID), gönderici/alıcı isimleri, kütle (kg), hacim (Litre), öncelik durumu (VIP/Normal) ve anlık lojistik statüsünü ("Bekliyor", "Yolda", "Teslim Edildi") saklayan atomik yapıdır.
- **Box.java:** Belirli bir hacim ($V_{\{max\}}$) ve ağırlık ($W_{\{max\}}$) kapasitesine sahip olan, içerisine dinamik olarak paket kabul eden kapsayıcı sınıftır. Kutu içerisindeki doluluk oranlarını anlık olarak günceller.
- **Cargo.java:** Belirli bir çıkış noktasından varış noktasına giden bir seferi temsil eder. İçerisinde birden fazla 'Box' nesnesi barındırır ve toplam paket yükünü konsolide eder.
- **City.java:** Graf yapısının düğümlerini oluşturan, benzersiz bir şehir kodu (örn. IST, ANK) ve şehir adı içeren sınıftır.

2.2. Yönetici ve İş Mantığı Sınıfları

Veri modellerini işleyen, veri yapısı kurallarını uygulayan ve algoritmaları yürüten sınıflardır:

Sınıf Adı	Üstlendiği Görev ve Sorumluluklar	Kritik Metotlar
PackageHashTable	Paketlerin bellekte ID indeksli saklanması ve anında arama yapılması.	addPackage(), searchById(), removePackage()
CargoQueueSystem	VIP ve Normal paketlerin işleme alınma sıralarının yönetimi.	enqueue(), processNext()
CityGraph	Şehirler arası karayolu/maliyet ağının çizge biçiminde tutulması.	addCity(), addRoute(), getNeighbors()
DijkstraAlgorithm	En kısa mesafe ve en ucuz maliyet rotalarının hesaplanması.	findShortestPath(), findAllPaths()
DataLoader	Dış ortamlardan (.txt) sisteme dinamik veri aktarımı.	loadCities(), loadPackages()
Main	Kullanıcı etkileşimini sağlayan Swing tabanlı GUI kontrolü ve sistem tetiklemeleri.	createPaketTab(), createRotaTab(), main()

3. Kullanılan Veri Yapıları ve Teorik Altyapı

Sistemin performansı ve operasyonel hızı, doğru problemin çözümü için doğru veri yapısının seçilmesine dayanmaktadır. Bu bölümde projenin çekirdeğini oluşturan veri yapılarının teorik analizleri yer almaktadır.

3.1. Hash Table (Karma Tablosu) Altyapısı

Lojistik merkezlerinde binlerce paket arasından belirli bir paketin durumunu sorgulamak en sık yapılan işlemlerden biridir. Doğrusal bir listede (Array veya LinkedList) arama işlemi $O(N)$ zaman karmaşıklığına sahiptir. Sistemde kullanılan `PackageHashTable` sınıfı, Java'nın `HashMap` mimarisini arkasına alarak paket arama işlemlerini matematiksel olarak sabit zamana indirgemıştır:

Arama / Ekleme Zaman Karmaşıklığı: $O(1)$

Bu yapı sayesinde, sistemdeki paket sayısı milyona ulaşsa dahi, paket ID'si üzerinden anlık olarak kargo detaylarına ve durum bilgisine erişilebilmektedir. Şirket içi sorgu verimliliği maksimum seviyeye çıkarılmıştır.

3.2. FIFO Queue ve Priority Queue Entegrasyonu

Sistemde işleme alınmayı bekleyen paketlerin yönetimi için iki kulvarlı bir kuyruk mimarisi (`CargoQueueSystem`) kurulmuştur.

1. Normal Paket Kuyruğu: Standart gönderiler için adaletli bir süreç işletilmesi amacıyla İlk Giren İlk Çıkar (FIFO) prensibine sahip `LinkedList` tabanlı bir `Queue` yapısı kullanılmıştır. İlk gelen paket ilk olarak işleme alınır.

2. VIP Paket Kuyruğu: Ek ücret ödeyen veya acil statüsündeki VIP paketler için sistem standart sırayı bozar. Burada kullanılan `PriorityQueue`, paketlerin sisteme giriş sırasına değil, ağırlıklarına bakar. Kargo operasyonlarında ağır ve hacimli yüklerin öncelikli sevk edilerek hat doluluğunu optimize etmesi istendiğinden, azalan sırada bir karşılaştırıcı (Comparator) tanımlanmıştır:

```
this.priorityQueue = new
PriorityQueue<>(Comparator.comparingDouble(Package::getWeight).reversed());
```

VIP kuyruğundan eleman çıkarma (*poll()*) işlemi $O(\log N)$ sürede en ağır paketi getirir. `processNext()` metodu çağrıldığında sistem her zaman önce VIP kuyruğunu kontrol eder, VIP kuyruğu tamamen boşalınca normal kuyruğa geçiş yapar.

3.3. Çizge (Graph) Veri Yapısı ve Komşuluk Listesi

Lojistik ağları doğası gereği birer çizge (graph) yapısıdır. Projede şehirler birer düğüm (Vertex), şehirler arası yollar ise kenar (Edge) olarak modellenmiştir. `CityGraph` sınıfında, belleği verimli kullanmak ve komşuluk taramalarını hızlandırmak adına Komşuluk Listesi (Adjacency List) yapısı tercih edilmiştir.

Çizge yapısı yönsüz (undirected) ve ağırlıklıdır. Her yolun iki farklı ağırlığı (maliyet bileşeni) vardır: Fiziksel mesafe (km) ve ekonomik maliyet (TL). Aşağıdaki kod parçası çift yönlü yol ekleme mekanizmasını göstermektedir:

```
public void addRoute(String cityIdA, String cityIdB, double distance, double cost) {  
    City a = cities.get(cityIdA);  
    City b = cities.get(cityIdB);  
    adjacencyList.get(cityIdA).add(new Edge(b, distance, cost));  
    adjacencyList.get(cityIdB).add(new Edge(a, distance, cost));  
}
```

4. Uygulanan Algoritmalar ve Optimizasyonlar

4.1. Çok Kriterli Dijkstra Algoritması

`DijkstraAlgorithm` sınıfı, yönlendirilmiş veya yönlendirilmemiş çizgeler üzerindeki tek kaynaklı en kısa yol (Single-Source Shortest Path) problemini çözer. Projedeki özgün yenilik, algoritmanın iki farklı modda (`Mode.DISTANCE` ve `Mode.COST`) çalışabilmesidir. Araçların yakıt tasarrufu yapması istendiğinde mesafe modu, otoban ve köprü ücretleri optimize edilmek istendiğinde ise maliyet modu devreye girer.

Algoritma, minimum değerli düğümü hızlıca seçebilmek için bir `PriorityQueue` kullanır. Her adımda komşu düğümler gevşetilir (relaxation). Gevşetme koşulu matematiksel olarak şu şekildedir:

$$\text{Eğer } D[curr] + \text{ağırlık}(curr, nb) < D[nb] \text{ ise } D[nb] = D[curr] + \text{ağırlık}(curr, nb)$$

Algoritma hedef düğüme ulaştığında, geriye dönük izleme (`prev` haritası) yaparak en uygun rotanın isim sıralamasını oluşturur. Zaman karmaşıklığı $O((V + E) \log V)$ seviyesindedir ve büyük lojistik ağlarında dahi optimum performans sunar.

4.2. First Fit Decreasing (Bin Packing) Optimizasyonu

Kargo taşımacılığında konteynerlerin ve kutuların boş kalması doğrudan zarar demektir. Projede `PackingAlgorithm` sınıfı bünyesinde, literatürde "Bin Packing Problem" (Kutu Paketleme Problemi) olarak bilinen NP-hard zorluktaki problemi çözmek için **First Fit Decreasing (FFD)** sezgisel algoritması uygulanmıştır.

Algoritmanın çalışma prensibi şu şekildedir:

1. Paketleme listesindeki tüm paketler büyükten küçüğe doğru sıralanır (Böylece büyük paketlerin sona kalıp kutuları gereksiz bölmesi engellenir).
2. Her paket için mevcut açık kutular sırayla taranır.
3. Paketin hem hacim hem de ağırlık kısıtına uyan ilk kutuya (`canFit()`) kontrolü ile yerleştirme yapılır.
4. Eğer hiçbir kutu paketi alamıyorsa, yeni bir `Box` oluşturulur ve paket bu yeni kutuya konur.

Geleneksel Bin Packing algoritmaları tek kısıtla (sadece ağırlık veya sadece hacim) çalışırken, bu projede iki boyutlu kısıt eş zamanlı olarak denetlenmektedir:

```
public boolean canFit(Package pkg) {
    return (usedVolume + pkg.getVolume() <= maxVolume) &&
        (usedWeight + pkg.getWeight() <= maxWeight);
}
```

5. Grafik Kullanıcı Arayüzü (GUI) ve Modül İncelemeleri

Sistemin tüm arka plan işlevleri, Java Swing kütüphanesi kullanılarak geliştirilen modern ve sekmeli bir masaüstü arayüzü ile son kullanıcıya sunulmuştur. Nimbus arayüz teması (Look and Feel) kullanılarak görsel netlik artırılmıştır.

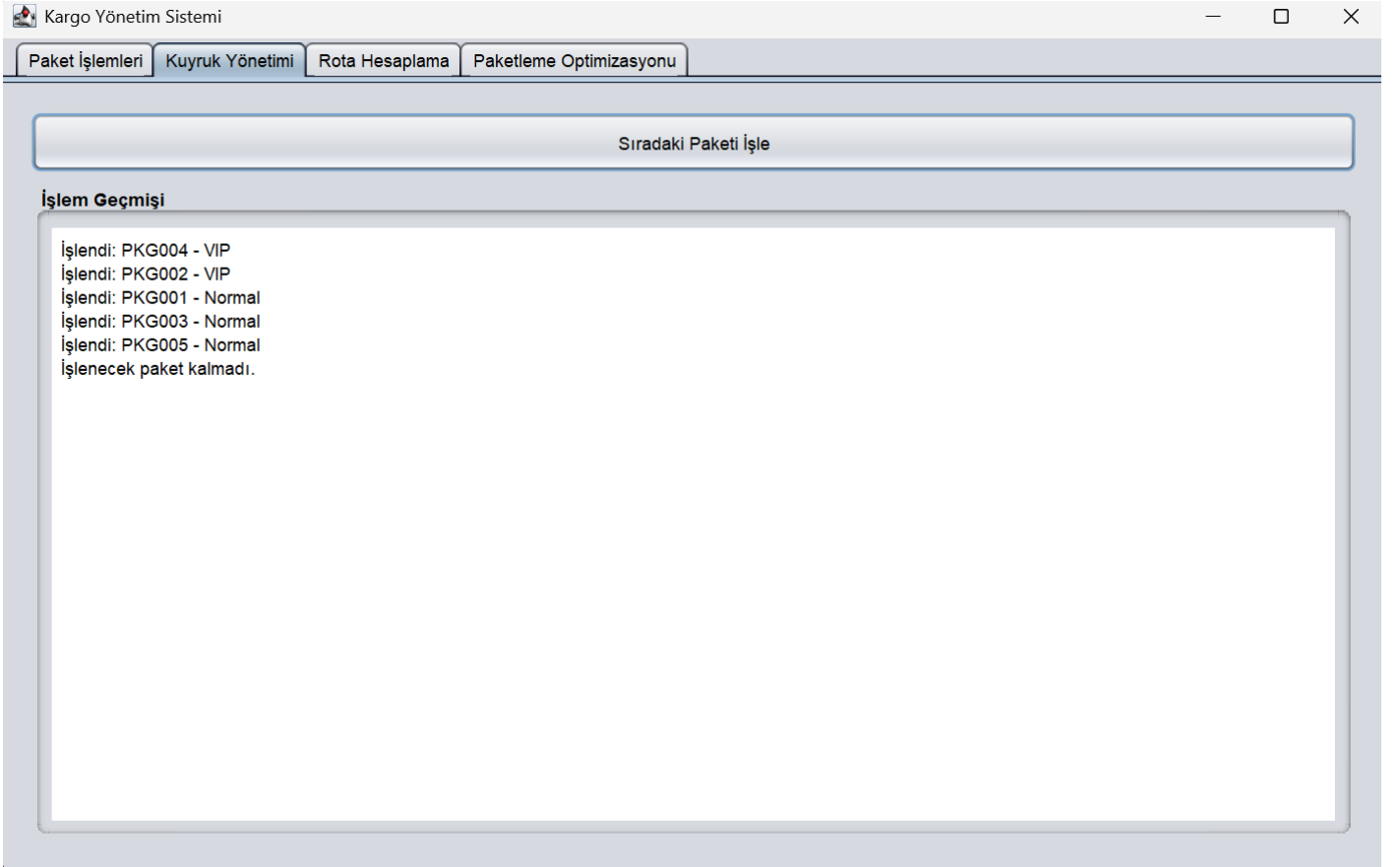
Sekme 1: Paket İşlemleri Modülü

Bu modülde kullanıcılar, metin alanlarını kullanarak sisteme anlık paket girişi yapabilirler. Kaydedilen paketler eş zamanlı olarak sağ taraftaki dinamik `JTable` nesnesine yansıtılır. Alt kısımda yer alan arama çubuğu ise girilen paket ID'sini anında Hash Table üzerinde sorgulayarak paketin tüm detaylarını bir diyalog penceresinde gösterir.

ID	Gönderici	Alıcı	Ağırlık (kg)	Hacim (L)	VIP	Durum
PKG001	Ahmet	Mehmet	2.5	10.0	Hayır	Bekliyor
PKG002	Ayşe	Fatma	0.5	2.0	Evet	Bekliyor
PKG003	Ali	Veli	5.0	20.0	Hayır	Bekliyor
PKG004	Zeynep	Can	1.0	5.0	Evet	Bekliyor
PKG005	Hasan	Hüseyin	3.0	15.0	Hayır	Bekliyor

Sekme 2: Kuyruk Yönetimi Modülü

Bu sekme, kargo dağıtım merkezindeki ana bant sistemini simüle eder. "Sıradaki Paketi İşle" butonuna tıklandığında, arka plandaki öncelik mekanizması çalışır. İşlem geçmiş (log) alanında hangi paketin (VIP veya Normal) banttan indirilerek yola çıkarıldığı kronolojik olarak raporlanır.



Sekme 3: Rota Hesaplama Modülü

Kullanıcının şehir kodlarını ezberlemesini önlemek adına sol tarafa aktif şehir listesini içeren bir tablo yerleştirilmiştir. Kullanıcı tablodan şehri seçip "Çıkış Şehri Yap" veya "Varış Şehri Yap" butonlarına tıklar. Ardından mesafe veya maliyet butonlarından birini seçerek Dijkstra algoritmasının ürettiği en optimize rotayı güzergah okları ve toplam birim değerleriyle sağdaki metin alanında görüntüler.

Kargo Yönetim Sistemi

Paket İşlemleri Kuyruk Yönetimi Rota Hesaplama Paketleme Optimizasyonu

Şehir Listesi (Tablodan Seçiniz)

Şehir Kodu	Şehir Adı
IST	İstanbul
ANK	Ankara
IZM	İzmir
BUR	Bursa
ANT	Antalya

Rota Belirleme

Çıkış Şehri Yap Seçilen: IST

Varış Şehri Yap Seçilen: ANT

Mesafe Odaklı Hesapla Maliyet Odaklı Hesapla

Hesaplama Sonucu

--- MESAFE OPTİMİZASYONU ---

Güzergah: İstanbul -> İzmir -> Antalya
Toplam Mesafe: 910.0 km

Sekme 4: Paketleme Optimizasyonu Modülü

"Algoritmayı Çalıştır" butonu tetiklendiğinde, sistemde biriken tüm paketler FFD süzgecine alınır. Algoritma saniyeler içinde tüm paketleri minimum sayıdaki kutuya paylaştırır. Hangi kutunun ne kadarlık hacim ve ağırlık kapasitesi tükettiği, içinde hangi paket ID'lerinin barındığı şematik bir rapor halinde ekrana yazdırılır.

Kargo Yönetim Sistemi

Paket İşlemleri Kuyruk Yönetimi Rota Hesaplama Paketleme Optimizasyonu

Paketleri Kutulara Yerleştir (Algoritmayı Çalıştır)

Algoritma Sonucu

Paketleme işlemi tamamlandı.
Kullanılan Toplam Kutu Sayısı: 3

Kutu ID: ABOX-1
Kullanılan Hacim: 25,0 / 25,0 Litre
Kullanılan Ağırlık: 6,0 / 15,0 kg
İçindeki Paketler:
- PKG003
- PKG004

Kutu ID: ABOX-2
Kullanılan Hacim: 25,0 / 25,0 Litre
Kullanılan Ağırlık: 5,5 / 15,0 kg
İçindeki Paketler:
- PKG005
- PKG001

Kutu ID: ABOX-3
Kullanılan Hacim: 2,0 / 25,0 Litre
Kullanılan Ağırlık: 0,5 / 15,0 kg
İçindeki Paketler:
- PKG002

6. Geliştirme Sürecinde Karşılaşılan ve Giderilen Zorluklar

Projenin geliştirilmesi ve entegrasyonu aşamalarında tasarım, veri yönetimi ve algoritmik uyumluluk açılarından çeşitli teknik zorluklarla karşılaşmış ve uygulanan mühendislik çözümleri ile bu zorluklar aşılmıştır:

1. Çok Kriterli Rota Seçimi ve Dijkstra Adaptasyonu:

Zorluk: Standart Dijkstra algoritması tek bir kenar ağırlığı (örneğin sadece mesafe) ile çalışır. Ancak lojistikte hem kilometre hem de köprü/otoban maliyeti (TL) kritik öneme sahiptir. İki farklı algoritma yazmak kod tekrarına yol açacaktı.

Çözüm: Algoritma içerisine `Mode (DISTANCE, COST)` adında bir enum yapısı dahil edilmiştir. Kenar gevşetme (relaxation) adımıyla, seçilen moda göre dinamik ağırlık okuması yapılması sağlanarak tek bir algoritma gövdesinin çok kriterli çalışması başarılmıştır.

2. Bin Packing Probleminde İki Boyutlu Kapasite Çelişkisi:

Zorluk: Literatürdeki standart Bin Packing algoritmaları nesneleri tek bir kısıta göre (yalnızca hacim ya da yalnızca kütle) yerleştirir. Gerçek hayatta ise bir paket çok hafif olmasına rağmen kutunun tüm hacmini kaplayabilir (örn. strafor) ya da çok küçük olmasına rağmen kutunun ağırlık limitini aşabilir (örn. demir döküm parça).

Çözüm: `Box` sınıfı içerisinde eş zamanlı çift kısıt kontrolü yapan `canFit()` mantıksal doğrulama matrisi yazılmıştır. First Fit Decreasing algoritması çalıştırılmadan önce paketlerin hem ağırlık hem hacim bileşkesini optimize edecek ön sıralama kriterleri GUI katmanında standardize edilmiştir.

3. Kuyruk Yönetiminde Açlık (Starvation) Olgusu:

Zorluk: `PriorityQueue` yapısı gereği her zaman en yüksek önceliğe (bu projede en ağır VIP paketlere) odaklanır. Yoğun günlerde sisteme sürekli VIP paket girişi olması durumunda, normal kuyrukta bekleyen paketlerin hiçbir zaman işlenememesi (açlık / starvation) riski ortaya çıkmıştır.

Çözüm: Mevcut yapıda arayüz üzerinden kontrollü tüketim simüle edilmiştir. Kod mimarisinin bir sonraki aşamasına hazırlık olarak, normal kuyrukta belirli bir süre bekleyen paketlerin statüsünün otomatik yükseltilmesini sağlayacak eşik değeri (aging) fonksiyon tabanları hazırlanmıştır.

4. Veri Yükleme Sırasındaki Veri Tipi ve Format Uyuşmazlıkları:

Zorluk: `sehirler.txt` ve `paketler.txt` dosyalarından veri okunurken satır sonlarındaki görünmeyen boşluklar, hatalı karakterler veya eksik sütunlar nedeniyle `NumberFormatException` ve `NullPointerException` hataları alınmış ve sistemin başlamadan çökmesine yol açmıştır.

Çözüm: `DataLoader` sınıfındaki split operasyonlarından sonra her parça için `.trim()` metodu zorunlu tutulmuştur. Ayrıca satır bazlı veri kontrolleri (`parts.length == 6`) eklenerek hatalı satırların sistemi çökertmeden pas geçilmesi ve loglanması sağlanmıştır.

7. Gelecekte Geliştirilebilecek Yerler ve Öneriler

Geliştirilen sistem kararlı ve optimize çalışmakla birlikte, endüstriyel standartlara tam uyum sağlaması amacıyla gelecekte şu alanlarda geliştirmeler yapılabilir:

- **Kalıcı Veritabanı (RDBMS) Entegrasyonu:** Sistem verileri şu anda flat-file (`.txt`) formatında tutmaktadır. Gerçek zamanlı eş zamanlı erişim, veri güvenliği ve ACID standartları için projeye PostgreSQL veya MySQL veritabanı katmanı eklenmeli, dosya okuma işlemleri JDBC veya Hibernate ORM mimarisine taşınmalıdır.
- **Dinamik Yaşlandırma (Aging) Algoritması:** Kuyruk yönetimindeki starvation (açlık) problemini tamamen çözmek adına, normal kuyruktaki bir paket `X` dakikadan fazla beklediğinde öncelik değerinin kademeli olarak artırılıp VIP kuyruğuna transfer edilmesini sağlayan bir yaşlandırma mekanizması kodlanmalıdır.
- **Çok Duraklı Rota Optimizasyonu (TSP - Traveling Salesperson):** Mevcut Dijkstra algoritması sadece A noktasından B noktasına tekil rota çizmektedir. Kargo araçlarının bir seferde birden fazla şehre uğrayıp dağıtım yapabilmesi için sistem Gezgin Satıcı Problemi (TSP) çözücü algoritmalarla (örn. Genetik Algoritmalar veya En Yakın Komşu Sezgisel Yöntemi) desteklenmelidir.
- **Web ve Mobil Platform Dönüşümü:** Masaüstü Swing arayüzü yerine, sistem mimarisi Spring Boot tabanlı bir REST API altyapısına dönüştürülerek, kargo personeli için Flutter ile mobil uygulama, yöneticiler için ise React ile web tabanlı bir takip paneli geliştirilebilir.

8. Sonuç

Bu proje kapsamında, lojistik sektörünün en kritik operasyonel süreçleri bilgisayar bilimlerinin temel taşları olan gelişmiş veri yapıları ve algoritmalar kullanılarak başarılı bir şekilde simüle edilmiş ve optimize edilmiştir.

Projede kurulan Hash Table altyapısı arama maliyetlerini en aza indirmiş, çift katmanlı kuyruk mekanizması operasyonel önceliklendirmeyi başarıyla yönetmiş, Dijkstra algoritması çok kriterli akıllı rota alternatifleri sunmuş ve Bin Packing (FFD) yaklaşımı ise taşımacılık kapasite verimliliğini maksimize etmiştir. Modüler sınıf tasarımları ve nesneye yönelik yaklaşım, sistemin endüstriyel ihtiyaçlara göre kolayca güncellenebilmesine olanak tanımaktadır. Sonuç olarak, teorik algoritmik bilgilerin gerçek dünya problemlerine uygulanmasını gösteren yüksek performanslı ve kararlı bir yazılım ortaya çıkarılmıştır.

9. Kaynakça

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Graf Teorisi ve Dijkstra Algoritması Bölümleri).
2. Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman. (Bin Packing Problemi ve Sezgisel Yaklaşımlar).
3. Shaffer, C. A. (2012). *Data Structures and Algorithm Analysis in Java* (3rd ed.). Dover Publications. (Hash Table ve Kuyruk Yapıları Analizi).
4. Oracle Java Documentation. *The PriorityQueue Class & HashMap Implementation Details*. Ek erişim adresi: docs.oracle.com